AI Models

My working understanding of what AI models are, how they differ, and where LLMs sit among them

Abhinav Ragidi

Started July 2026

Contents

What an AI Model Actually Is

> **What this chapter covers:** the single idea underneath every AI model, regardless of family — and the one distinction (training vs inference) that clears up most confusion.

Topic started July 2026. This is the parent note for the AI-models track; LLMs get their own deep-dive in `llms/`.

The one-sentence definition

An AI model is a function whose behaviour was learned from data rather than written by hand.

That is the whole idea. Everything else — architectures, model families, buzzwords — is variation on how the function is shaped and how the learning happens.

Compare the two ways to build the same thing:

	Traditional program	AI model
Where the logic comes from	A human writes the rules	The rules are fitted to examples
What you author	The algorithm	The data, the objective, and the shape of the model
What it produces	Exact, repeatable output	A prediction, with a confidence
Fails by	Crashing or throwing	Being *confidently wrong*
Debugging	Read the code	Inspect the data and the outputs

That last row is the one that matters in practice. A traditional program tells you when it breaks. A model just returns something plausible. Every serious design decision downstream — validation, evals, guardrails — exists because of it.

Three things every model has

1. **Parameters (weights)** — the learned numbers. This *is* the model. Everything the model "knows" lives here, as billions of floats with no human-readable structure.
2. **Architecture** — the wiring that decides how inputs flow through those parameters. CNN, transformer, decision tree. Chosen by a human, not learned.
3. **An objective (loss function)** — the definition of "wrong". Training is nothing but the search for parameters that reduce this number.

A model is a **compressed, lossy summary of its training data, shaped by its architecture and pointed at by its objective.** Hold that sentence and most model behaviour becomes predictable rather than mysterious.

Training vs inference — the distinction to get right first

These are the two completely different modes a model lives in, and conflating them causes real confusion (especially about cost, and about whether a model "learns" from you).

Training — slow, offline, done once, enormously expensive. Data goes in, the loss is computed, gradients flow backwards, weights get nudged. Repeat millions of times.

Inference — fast, online, per-request, comparatively cheap. **The weights are frozen.** Input goes in one direction through the network, a prediction comes out. Nothing is written back.

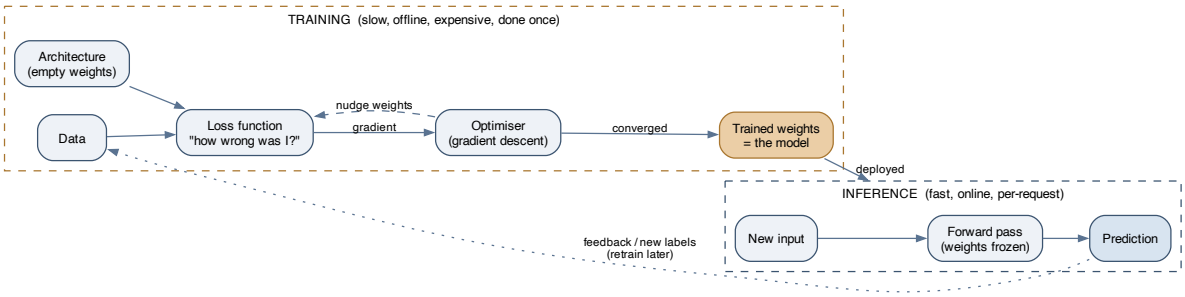

Figure 1: Training produces the weights; inference consumes them. The weights are frozen during inference — a model does not learn from your requests. Any apparent learning is either a new training run, or context you supplied in the request itself.

Two consequences worth internalising:

- **A model does not remember you.** If a chatbot "remembers" what you said earlier, that history was re-sent to it as input. There is no persistent state inside the model.
- **A model's knowledge has a cutoff.** Its facts are whatever was in the training data at training time. Newer facts must be *supplied* at inference time — this is the whole justification for retrieval and tool use.

Supervised, unsupervised, self-supervised, reinforcement

How you get a learning signal:

Paradigm	The signal	Typical use
Supervised	Human-provided labels: (input, correct answer)	Classification, regression. Accurate but labels are expensive
Unsupervised	No labels — find structure in the data	Clustering, dimensionality reduction, anomaly detection

Paradigm	The signal	Typical use
Self-supervised	The data labels itself: hide part of the input, predict it	**How LLMs are pretrained.** Unlocks web-scale data with no annotation cost
Reinforcement	A reward signal from acting in an environment	Games, robotics, and preference-tuning LLMs (RLHF)

Self-supervised learning is the unlock behind the last few years. Supervised learning is bounded by how much data humans will label. Self-supervision has no such ceiling — "predict the next token" turns the entire internet into a labelled dataset for free. That is why models suddenly got so much bigger and so much more capable.

Discriminative vs generative

A cut that predicts how a model will be used:

- **Discriminative** models learn `P(y | x)` — given this input, which label? Spam detection, credit scoring, object detection. Narrow, efficient, easy to evaluate.
- **Generative** models learn `P(x)` — what does real data look like? — so they can produce new samples. LLMs, diffusion models, TTS.

The asymmetry is useful: a generative model can do a discriminative task (ask an LLM to classify), but a classifier can't write you an essay. This is why generative foundation models displaced so many task-specific classifiers — one general model, adapted many ways, often beats a fleet of narrow ones on everything except cost and latency.

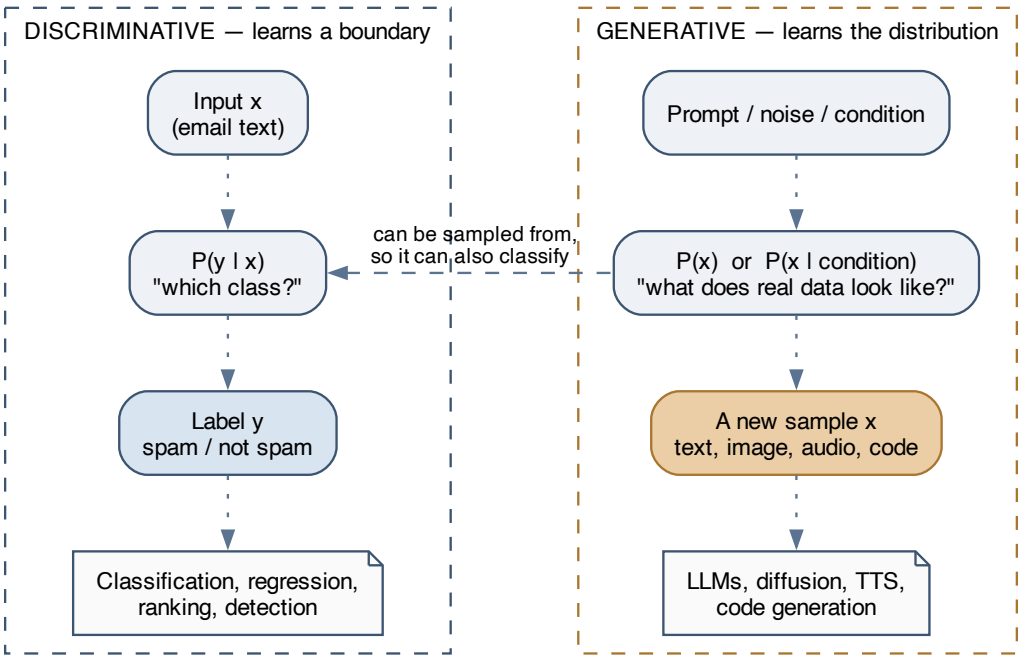

Figure 2: A discriminative model learns a boundary between classes; a generative model learns what the data itself looks like, so it can produce new samples. Generative models can be used discriminatively, but not the reverse.

The Model Families

What this chapter covers: a map of the main architecture families, what each is good at, and why the transformer swallowed most of the field.

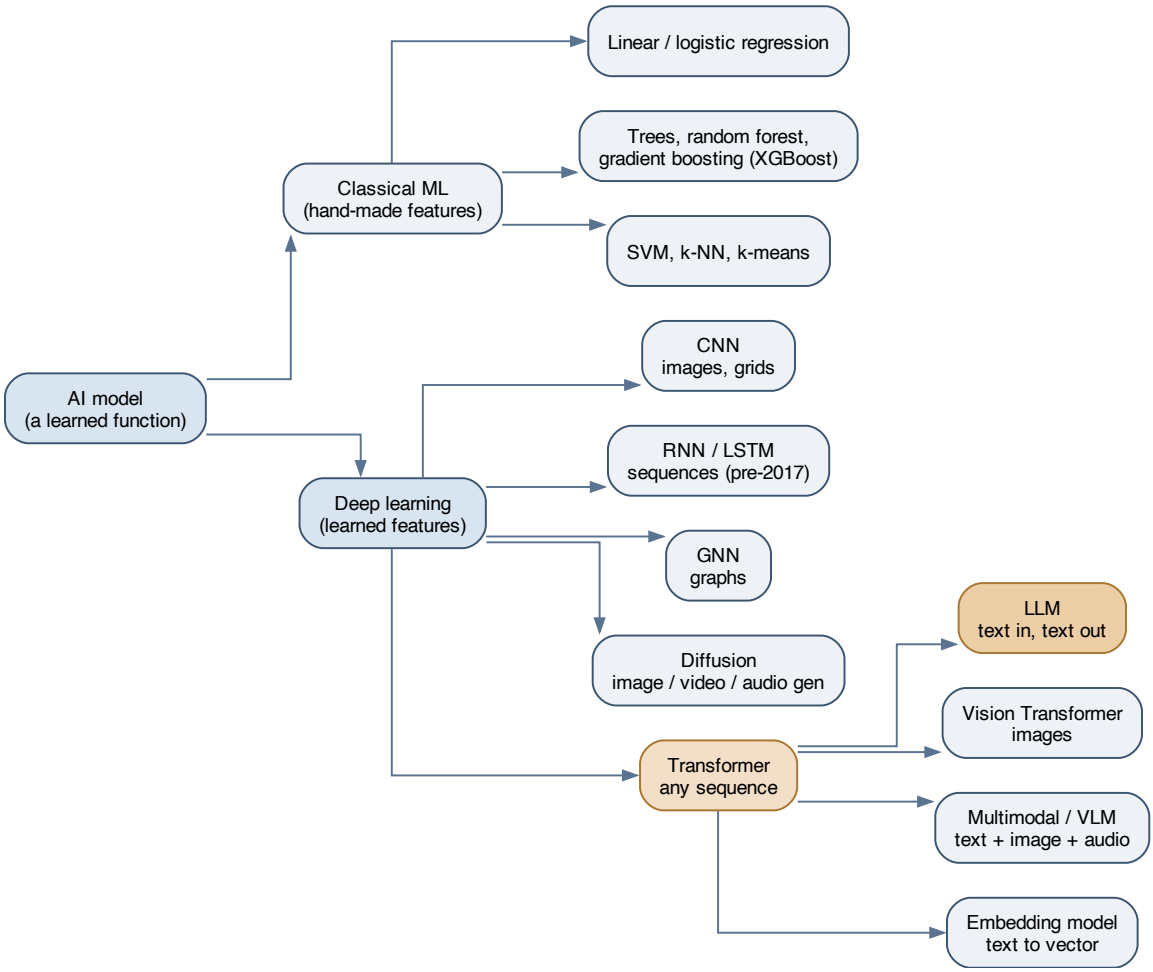

Figure 3: The families, from classical ML through deep learning to the transformer branch. LLMs are one leaf on the transformer branch — the same architecture also produces vision models, multimodal models, and embedding models.

Classical ML — still the right answer more often than people admit

Linear and logistic regression, decision trees, random forests, gradient boosting (XGBoost, LightGBM), SVMs, k-means.

The defining property: **a human designs the features.** You decide that "number of exclamation marks" and "sender age in days" are what matter; the model learns how to weigh them.

Where these still win outright:

- **Tabular data.** Gradient-boosted trees remain extremely competitive on structured rows-and-columns problems. Reaching for a neural net here is usually a mistake.
- **Small datasets.** Deep learning is data-hungry; these are not.
- **Interpretability requirements.** You can read a decision tree. You cannot read 70 billion weights.
- **Latency and cost budgets** measured in microseconds and fractions of a cent.

Deep learning — the model learns the features too

Stacked layers of simple units, where each layer learns a progressively more abstract representation. Early layers of an image model find edges; middle layers find textures; late layers find faces. Nobody specified "edge" — it emerged because it was useful for reducing the loss.

This is the actual leap: feature engineering, previously the bulk of the human work, moved inside the model.

Family	Built for	Key idea
CNN	Images, grids	Slide a small filter over local neighbourhoods; share those weights everywhere
RNN / LSTM	Sequences (the pre-2017 answer)	Carry a hidden state forward, one step at a time
GNN	Graphs — molecules, social networks	Pass messages along edges between nodes
Diffusion	Image, video, audio generation	Learn to denoise; then generate by denoising pure noise, step by step
Transformer	Any sequence	Attention: let every position look at every other position at once

Why the transformer took over

RNNs process a sequence one step at a time, which creates two problems: it can't be parallelised (step n needs step $n-1$), and information from early in the sequence gets diluted by the time you reach the end.

Attention fixed both. Every position attends to every other position **in a single parallel operation**. Long-range dependencies become a direct lookup rather than a long chain of hand-offs, and the whole thing saturates a GPU.

The consequences went beyond quality:

- **It scales predictably.** More parameters plus more data plus more compute reliably yields better models — a relationship stable enough to plan budgets around.
- **It generalises across modalities.** Text, images (as patch sequences), audio, protein sequences, code. One architecture, many domains — which is why the field consolidated instead of fragmenting further.

The cost: attention compares every token to every other token, so compute grows quadratically with sequence length. **That quadratic cost is the direct reason context windows are finite** — and a large part of why long-context inference is expensive.

Foundation models — the current shape of the field

The economics inverted. Pretraining is so expensive that only a handful of organisations do it, and the resulting model is so general that everyone else adapts it rather than training their own.

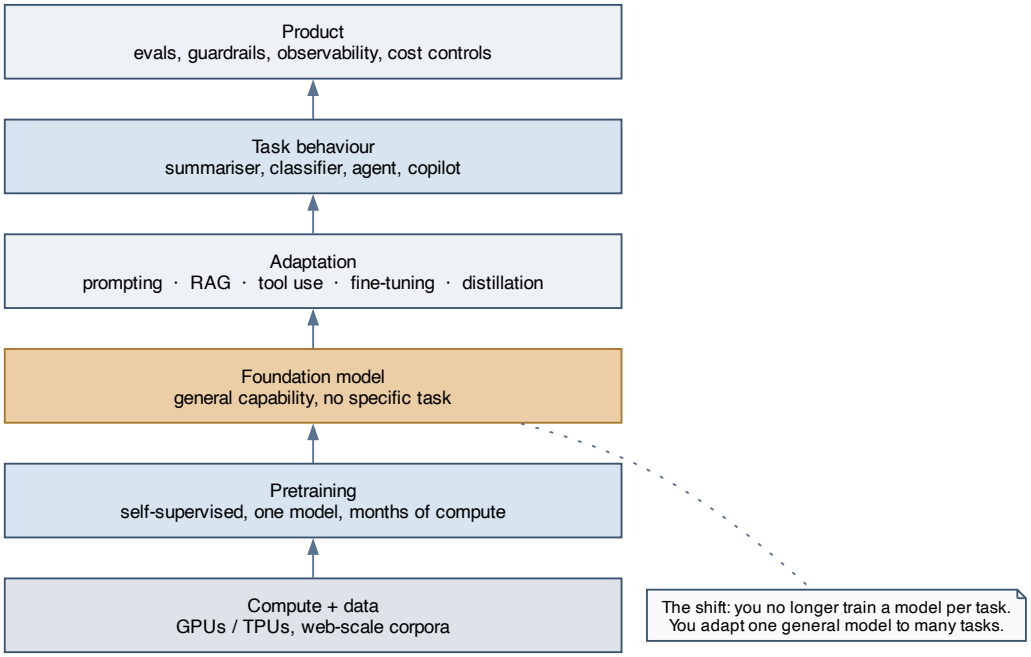

Figure 4: The foundation-model stack. Pretraining happens once, upstream, by someone else. Everything you do lives in the adaptation layer and above.

The old workflow was *collect labels for your task, train a model for your task.* The new one is *take a general model, adapt it to your task.* The skills that matter shifted accordingly — away from architecture design, toward prompting, retrieval, evaluation, and orchestration.

Choosing a model — the questions in order

1. **What's the output shape?** A label, a number, a ranking, or freely-generated content? Generation means a generative model; the rest usually don't.
2. **What does the input look like?** Tabular → gradient boosting. Images → CNN or ViT. Text or sequences → transformer. Graphs → GNN.
3. **How much labelled data do I have?** Very little → a pretrained model with prompting or light fine-tuning. A lot, and tabular → classical ML.
4. **What are my latency and cost budgets?** This constraint eliminates more options than capability does. A large model that can't hit your p99 is not a candidate.
5. **Do I need to explain a decision?** Regulated domains may rule out the whole deep-learning branch regardless of accuracy.
6. **How bad is a wrong answer, and can I catch it?** This sets how much validation and human review the pipeline needs — see the LLM note, where it becomes the dominant design driver.

The bias worth having: start with the simplest model that could work, and make something more complex earn its place with a measured improvement.

Where LLMs Fit

An LLM is one specific leaf: **a generative, self-supervised, transformer-based, autoregressive model over text tokens.** Each of those words is a choice, and each has been made differently elsewhere on the tree.

They deserve their own note for two reasons:

1. **They're general-purpose in a way other models aren't.** A spam classifier does one thing. An LLM does summarisation, extraction, classification, translation, code generation, and dialogue — from the same weights, selected by prompt. The engineering problem becomes *how do I reliably get the behaviour I want out of a general system*, which is a different problem from *how do I train a model for this task*.
2. **Their failure mode is unusual.** Most models fail visibly — low confidence, out-of-distribution warnings. An LLM fails *fluently*. Fluency and correctness are separate axes, and nothing in the output signals which one you got.

Continue in `llms/llms.md` — covering tokenisation, attention, the training stages, decoding, and, at length, how observable model behaviour should drive pipeline design.

Vocabulary I want to keep straight

Term	What it means
Parameters / weights	The learned numbers. "70B parameters" = 70 billion of them
Epoch	One full pass over the training data
Overfitting	Memorised the training set; fails on new data. The core failure of training
Generalisation	Performance on data never seen. The only thing that actually matters
Inductive bias	Assumptions baked into the architecture (a CNN assumes nearby pixels relate)
Embedding	A dense vector representing something; nearby vectors mean similar things
Fine-tuning	Continuing training on a pretrained model with your own narrower data
Distillation	Training a small model to imitate a large one — most of the quality, less cost
Quantisation	Storing weights at lower precision to cut memory and increase speed

Term	What it means
Zero-shot / few-shot	Doing a task with no examples / with a handful of examples in the prompt

Key takeaways

One-line summary. An AI model is a function learned from data; the family you pick follows from your input shape, data volume, and latency budget; transformers won because attention parallelises and generalises across modalities; and the current era is defined by adapting someone else's pretrained foundation model rather than training your own.

Questions to revisit:

- Where exactly does gradient boosting still beat a fine-tuned transformer on tabular data, and by how much?
- How do diffusion and autoregressive generation compare when both can do the same task?
- When is distillation into a small model the right call over routing to a large one?